

Milestone 3 | Week 3 | CSS Interview Questions

Q1. What is normal flow in CSS? Why is it important to understand before learning layouts?

Answer: Normal flow is the default way a browser arranges HTML elements on a page — without any CSS layout applied.

- **Block elements** (like `div`, `p`, `h1`) stack vertically, one below the other, taking full width.
- **Inline elements** (like `span`, `a`) sit side by side within a line of text.

Why it matters: If you don't understand normal flow, you won't understand *why* your layout breaks when you start adding positioning or flex. Every CSS layout property changes behavior *relative to* normal flow.

```
<div>Block 1</div>    <!-- takes full width, new line -->
<div>Block 2</div>    <!-- stacks below Block 1 -->
<span>Inline A</span> <span>Inline B</span> <!-- sit side by side -->
```

Interview Tip: Interviewers often ask this to check if you truly understand CSS fundamentals or just memorized properties.

Q2. What is the `position` property in CSS? What are its main values?

Answer: The `position` property controls how an element is placed in the document.

Value	Behavior
<code>static</code>	Default. Normal flow. No special placement.
<code>relative</code>	Moves from its original position. Still occupies original space.
<code>absolute</code>	Removed from flow. Positioned relative to nearest positioned parent.

Value	Behavior
<code>fixed</code>	Removed from flow. Stays fixed relative to the browser window.
<code>sticky</code>	Stays in flow, but sticks at a certain scroll position.

```
.box {
  position: relative;
  top: 20px; /* moves 20px down from its original spot */
}
```

Q3. A fresher at Flipkart was asked to add a "SALE" badge on a product card. The badge kept appearing in the wrong place. What CSS concept should they use?

Answer: Use `position: absolute` on the badge and `position: relative` on the parent product card.

```
/* ✅ Correct approach */
.product-card {
  position: relative; /* this is the reference point */
}

.badge {
  position: absolute;
  top: 10px;
  right: 10px;
  background-color: red;
  color: white;
  padding: 4px 8px;
  border-radius: 4px;
}

<div class="product-card">
  
  <span class="badge">SALE</span>
</div>
```

Common Mistake: Forgetting to add `position: relative` to the parent. The badge then jumps to an unexpected location on the page.

Q4. What is the difference between `position: relative` and `position: absolute`?

Answer:

Feature	relative	absolute
Stays in normal flow?	✅ Yes — original space is reserved	❌ No — removed from flow
Positioned relative to	Its own original position	Nearest positioned ancestor
Use case	Small shifts, reference parent	Badges, tooltips, dropdowns

```
/* relative - shifts but still holds its space */
.relative-box {
  position: relative;
  top: 10px;
  left: 20px;
}

/* absolute - floats freely inside positioned parent */
.absolute-box {
  position: absolute;
  top: 0;
  right: 0;
}
```

Q5. A Zomato intern built a navigation bar that scrolls away when users scroll down. The manager asked them to keep the navbar always visible. What CSS property should they use?

Answer: Use `position: fixed`.

```
/* ✅ Fixed navbar */
.navbar {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  background-color: #ffffff;
  z-index: 1000;
  padding: 16px 24px;
```

```
box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

body {
padding-top: 64px; /* same as navbar height so content doesn't hide behind it */
}
```

Interview Tip: Always add `padding-top` to the body when using a fixed navbar, otherwise content will hide under it.

Q6. What is `z-index` in CSS? When would you use it?

Answer: `z-index` controls the stacking order of overlapping elements. Higher value = appears on top.

Rules:

- Only works on elements with `position` other than `static`
- Default `z-index` is `auto` (treated as 0)

```
.background-image {
position: absolute;
z-index: 1;
}

.popup-modal {
position: fixed;
z-index: 999; /* sits on top of everything */
}
```

Real scenario: On Swiggy's website, the "Add to Cart" popup should appear above the food listing — use `z-index` to achieve this.

Common Mistake: Setting `z-index` on a `position: static` element — it has no effect.

Q7. What is the `overflow` property? Give a real example of when you would use it.

Answer: `overflow` controls what happens when content is too large to fit inside its container.

Value	Behavior
visible	Content spills out (default)
hidden	Overflow is cut off and hidden
scroll	Adds scrollbar always
auto	Adds scrollbar only when needed

/ ❌ Bad — text spills out of the box */*

```
.description {
  height: 100px;
  overflow: visible;
}
```

/ ✅ Good — scrollable description box */*

```
.description {
  height: 100px;
  overflow-y: auto;
}
```

Real use: Myntra's product description box — scrollable when the description is too long.

Q8. What is Flexbox? Why was it introduced in CSS?

Answer: Flexbox (Flexible Box Layout) is a CSS layout system that makes it easy to arrange items in a row or column and align them properly — even when their sizes change.

Before Flexbox, developers used floats and hacks to align content. These were hard to use and broke easily on different screen sizes.

Flexbox solves:

- Centering elements vertically and horizontally
- Distributing space between items
- Reordering elements without changing HTML

```
.container {
  display: flex; /* activates flexbox on the parent */
}
```

Once you write `display: flex`, all direct children automatically become flex items arranged in a row.

Q9. What is the difference between a flex container and a flex item?

Answer:

Term	What it is	CSS applied to
Flex Container	The parent that has <code>display: flex</code>	The parent/wrapper
Flex Item	Direct children of the flex container	The children inside

```
<div class="container">      <!-- flex container -->
  <div class="item">A</div>    <!-- flex item -->
  <div class="item">B</div>    <!-- flex item -->
  <div class="item">C</div>    <!-- flex item -->
</div>

.container {
  display: flex;
}
```

Interview Tip: Properties like `justify-content` and `align-items` go on the **container**. Properties like `flex-grow` and `align-self` go on the **item**.

Q10. A Naukri.com developer needs to display job category cards side by side. How would they use Flexbox?

Answer:

```
<div class="categories">
  <div class="card">IT Jobs</div>
  <div class="card">Finance Jobs</div>
  <div class="card">Design Jobs</div>
</div>

.categories {
```

```

    display: flex;
    gap: 16px;
}

.card {
    background: #f0f4ff;
    padding: 20px;
    border-radius: 8px;
    flex: 1; /* equal width for each card */
}

```

Q11. What is `justify-content` in Flexbox? Explain with a real example.

Answer: `justify-content` aligns flex items along the **main axis** (horizontal when `flex-direction: row`).

Value	Effect
<code>flex-start</code>	Items start from left (default)
<code>flex-end</code>	Items pushed to right
<code>center</code>	Items centered
<code>space-between</code>	Equal space between items, no edge space
<code>space-around</code>	Equal space around each item
<code>space-evenly</code>	Equal space everywhere

```

/* Swiggy navbar - logo left, links right */
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 0 24px;
}

```

Q12. What is `align-items` in Flexbox? How is it different from `justify-content`?

Answer:

Property	Axis	Controls
<code>justify-content</code>	Main axis (horizontal by default)	Horizontal alignment
<code>align-items</code>	Cross axis (vertical by default)	Vertical alignment

```
.container {  
  display: flex;  
  justify-content: center; /* center horizontally */  
  align-items: center;    /* center vertically */  
  height: 300px;  
}
```

Very Common Interview Question: "How do you center a div?" — Answer:

`display: flex; justify-content: center; align-items: center;`
on the parent.

Q13. What is the main axis and cross axis in Flexbox?

Answer:

- **Main axis** = direction items are placed (set by `flex-direction`)
- **Cross axis** = perpendicular to the main axis

<code>flex-direction</code>	Main Axis	Cross Axis
<code>row</code> (default)	Horizontal →	Vertical ↓
<code>column</code>	Vertical ↓	Horizontal →

- `justify-content` works on the **main axis**
- `align-items` works on the **cross axis**

Interview Tip: This confuses many beginners. If you can explain main vs cross axis clearly, you will impress interviewers.

Q14. A TCS developer was building a login page and the form stayed stuck to the top-left. How would they center it using Flexbox?

Answer:

```
body {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
  background-color: #f5f5f5;
}

.login-form {
  background: white;
  padding: 40px;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  width: 360px;
}
```

Q15. A fresher noticed `align-items: center` was not working. What is the most common reason?

Answer: `align-items` centers on the cross axis (vertically, in a row layout). For it to work, the container must have a defined height — without height, there is no space to center within.

/ ❌ Bad — no height, so nothing to center in */*

```
.container {
  display: flex;
  align-items: center;
}
```

/ ✅ Good — height gives space for vertical centering */*

```
.container {
  display: flex;
  align-items: center;
  height: 300px;
}
```

Q16. What does `flex-wrap` do? When should you use it?

Answer: By default, flex items try to squeeze into one line. `flex-wrap: wrap` allows items to move to the next row when they don't fit.

```
/* ✅ Cards wrap to next row on smaller screens */
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}

.card {
  flex: 1 1 250px; /* minimum width 250px, then wraps */
}
```

Real use: Myntra's product listing page — cards wrap to the next row instead of squeezing into one line.

Q17. What is `flex-grow` and how does it work?

Answer: `flex-grow` controls how much extra space a flex item takes compared to others.

```
.container {
  display: flex;
}

.sidebar {
  flex-grow: 0;
  width: 200px; /* fixed size */
}

.main-content {
  flex-grow: 1; /* takes all remaining space */
}
```

Real use: On a Zoho dashboard — sidebar is fixed, main content fills the rest.

Q18. How do you build a footer that always stays at the bottom of the page, even with little content?

Answer:

```

body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  margin: 0;
}

main {
  flex: 1; /* pushes footer down */
}

footer {
  background: #333;
  color: white;
  padding: 24px;
  text-align: center;
}

```

Interview Tip: This is a very common practical problem. Solving it cleanly with Flexbox shows real project experience.

Q19. A Wipro developer is building a three-column layout. How would they do it with Flexbox?

Answer:

```

<div class="layout">
  <aside class="sidebar-left">Left</aside>
  <main class="main-content">Main</main>
  <aside class="sidebar-right">Right</aside>
</div>

.layout {
  display: flex;
  gap: 16px;
  min-height: 100vh;
}

.sidebar-left,
.sidebar-right {
  flex: 0 0 240px; /* fixed 240px */
}

.main-content {

```

```
flex: 1;           /* takes remaining space */  
}
```

Q20. What is **gap in Flexbox? Why is it better than using **margin**?**

Answer:

/* ❌ Old — margin adds unwanted space on last item edge */

```
.item {  
  margin-right: 16px;  
}
```

/* ✅ Better — gap adds space between items only */

```
.container {  
  display: flex;  
  gap: 16px;  
}
```

gap is cleaner, doesn't create extra space at the edges, and works for both Flexbox and Grid.

Q21. What is responsive design? Why does it matter for Indian web projects?

Answer: Responsive design means a website works well and looks good on all screen sizes — mobile, tablet, and desktop — without needing a separate mobile website.

Why it matters in India:

- Over 75% of Indian internet users browse on mobile phones
- Government portals, e-commerce apps like Flipkart and Meesho, and job portals like Naukri are all used heavily on phones
- A site that breaks on mobile loses real users and business

How it is achieved:

- Flexible layouts using Flexbox and Grid
- Media queries to adjust styles at different screen widths
- Relative units like %, vw, rem instead of fixed px

Q22. What is a media query in CSS? Write a basic example.

Answer: A media query applies CSS styles only when a specific condition is true — most commonly, when the screen is a certain width.

```
/* Styles here apply to ALL screen sizes */
.container {
  font-size: 16px;
  padding: 24px;
}

/* Styles inside this block apply ONLY when screen is 768px or wider */
@media (min-width: 768px) {
  .container {
    font-size: 18px;
    padding: 40px;
  }
}
```

Syntax:

```
@media (condition) {
  /* CSS rules */
}
```

Interview Tip: Media queries are one of the most asked topics in any frontend interview. Know the syntax by heart.

Q23. What is the difference between `min-width` and `max-width` in media queries?

Answer:

	<code>min-width</code>	<code>max-width</code>
Applies styles when	Screen is at least that wide	Screen is at most that wide
Used in	Mobile-first approach	Desktop-first approach
Example	Tablet and desktop screens	Mobile and small screens

```
/* min-width - styles for screens 768px and WIDER */
@media (min-width: 768px) {
  .cards {
    display: grid;
  }
}
```

```

        grid-template-columns: repeat(3, 1fr);
    }
}

/* max-width - styles for screens 767px and NARROWER */
@media (max-width: 767px) {
    .cards {
        display: block;
    }
}

```

Q24. What is mobile-first design? Why is it recommended?

Answer: Mobile-first means you write your base CSS for small screens first, then use **min-width** media queries to add styles for larger screens.

```

/* Base styles - mobile (small screens) */
.card {
    width: 100%;
    font-size: 14px;
}

/* Tablet and up */
@media (min-width: 768px) {
    .card {
        width: 48%;
        font-size: 16px;
    }
}

/* Desktop and up */
@media (min-width: 1024px) {
    .card {
        width: 32%;
    }
}

```

Why recommended:

- Most Indian users are on mobile — design for them first
- Easier to scale up than to strip down desktop styles
- Better performance on slow mobile networks

Interview Tip: Interviewers love asking "what is mobile-first?" — a clear answer with code shows senior-level thinking.

Q25. A developer at a startup built a product page with fixed **px widths. On mobile it was broken. What is the fix?**

Answer:

/ ❌ Bad — fixed pixel widths break on small screens */*

```
.product-grid {  
  width: 1200px;  
  display: flex;  
}
```

```
.product-card {  
  width: 300px;  
}
```

/ ✅ Good — flexible layout with media queries */*

```
.product-grid {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 16px;  
  max-width: 1200px;  
  width: 100%;  
  margin: 0 auto;  
}
```

```
.product-card {  
  flex: 1 1 250px; /* flexible, wraps on small screens */  
}
```

```
@media (max-width: 600px) {  
  .product-card {  
    flex: 1 1 100%; /* full width on mobile */  
  }  
}
```

Q26. What are breakpoints in CSS? What are the most commonly used breakpoints?

Answer: Breakpoints are specific screen widths where your layout changes to better fit the screen size.

Commonly used breakpoints:

Breakpoint	Screen Size	Device
< 576px	Extra small	Small phones
576px	Small	Large phones
768px	Medium	Tablets
1024px	Large	Laptops
1280px	Extra large	Desktops

```
/* Mobile first */
.container { width: 100%; }

@media (min-width: 768px) {
  .container { max-width: 768px; }
}

@media (min-width: 1024px) {
  .container { max-width: 1024px; }
}
```

Interview Tip: Don't memorize too many breakpoints. Most real projects use 2-3 key breakpoints.

Q27. What is CSS Grid? How is it different from Flexbox?

Answer: CSS Grid is a **2D layout system** — it controls both rows and columns at the same time. Flexbox is **1D** — it controls either a row or a column at a time.

Feature	Flexbox	CSS Grid
Dimensions	1D (row or column)	2D (rows AND columns)
Best for	Navigation bars, card rows, alignment	Page layouts, dashboards, complex grids
Control	Items control their own sizing	Container controls the layout


```

/* Flexbox - items in a row */
.flex-container {
  display: flex;
}

/* Grid - rows AND columns defined on container */
.grid-container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr; /* 3 equal columns */
}

```

Interview Tip: "Use Flexbox for components, use Grid for page layout" is a good rule of thumb.

Q28. What is `grid-template-columns`? Write an example.

Answer: `grid-template-columns` defines how many columns the grid has and how wide each column is.

```

/* 3 equal columns */
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}

/* Shorthand - same as above */
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

/* Mixed - fixed sidebar + flexible main */
.layout {
  display: grid;
  grid-template-columns: 240px 1fr;
}

```

`1fr` means "1 fraction of available space." If you have `repeat(3, 1fr)`, each column gets one-third of the total width.

Q29. What is **fr** unit in CSS Grid? Why is it useful?

Answer: **fr** stands for **fraction**. It divides the available space into equal (or proportional) parts.

```
/* All 3 columns equal width */
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}

/* First column twice as wide */
.grid {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr;
}

/* Fixed + flexible */
.layout {
  display: grid;
  grid-template-columns: 200px 1fr;
  /* sidebar: 200px, main: takes rest */
}
```

Why useful: **fr** automatically adjusts to screen size — much better than calculating percentages manually.

Q30. A developer at Infosys wants to display a 3-column product grid using CSS Grid. How would they do it?

Answer:

```
<div class="product-grid">
  <div class="product-card">Product 1</div>
  <div class="product-card">Product 2</div>
  <div class="product-card">Product 3</div>
  <div class="product-card">Product 4</div>
  <div class="product-card">Product 5</div>
  <div class="product-card">Product 6</div>
</div>

.product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 24px;
```

```

padding: 24px;
}

.product-card {
  background: white;
  border: 1px solid #e0e0e0;
  border-radius: 8px;
  padding: 16px;
}

/* Responsive - 1 column on mobile */
@media (max-width: 600px) {
  .product-grid {
    grid-template-columns: 1fr;
  }
}

/* 2 columns on tablet */
@media (min-width: 601px) and (max-width: 1023px) {
  .product-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

```

Q31. What is `grid-template-rows`? Do you always need to define it?

Answer: `grid-template-rows` defines the height of rows in a grid, similar to how `grid-template-columns` defines column widths.

```

.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: 200px auto;
  /* Row 1: 200px tall, Row 2: auto height */
}

```

Do you always need it? No. If you don't define rows, the browser automatically creates rows as needed — called **implicit rows**. For most layouts, you only define columns and let rows auto-generate.

Q32. What is `gap` in CSS Grid? Is it different from `gap` in Flexbox?

Answer: `gap` works the same way in both Grid and Flexbox — it adds space between items (not at the edges).

```
/* Grid gap */
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;          /* same gap in all directions */
}

/* Or control separately */
.grid {
  row-gap: 20px;      /* vertical space between rows */
  column-gap: 16px;   /* horizontal space between columns */
}
```

The behavior is identical in both Flexbox and Grid — it is the same property.

Q33. How would you build a full page layout with a header, sidebar, main content, and footer using CSS Grid?

Answer:

```
<div class="page-layout">
  <header>Header</header>
  <aside>Sidebar</aside>
  <main>Main Content</main>
  <footer>Footer</footer>
</div>

.page-layout {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: 64px 1fr 60px;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  min-height: 100vh;
}

header { grid-area: header; background: #1a1a2e; color: white; }
aside { grid-area: sidebar; background: #f0f0f0; }
```

```
main    { grid-area: main;    padding: 24px; }
footer  { grid-area: footer;  background: #333; color: white; }
```

grid-template-areas lets you name regions — making the layout easy to read and change.

Q34. What is **auto-fill** vs **auto-fit** in CSS Grid?

Answer: Both are used with **repeat()** to automatically create columns that fit the container.

```
/* auto-fill - creates as many columns as possible, even empty ones */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
}

/* auto-fit - same, but collapses empty columns so items stretch */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
}
```

Real use: A Zomato restaurant card grid that automatically shows 4 cards on desktop, 2 on tablet, 1 on mobile — without writing any media queries.

Interview Tip: **repeat(auto-fit, minmax(200px, 1fr))** is one of the most powerful one-line responsive layouts in CSS.

Q35. What is a CSS transition? How is it different from an animation?

Answer:

Feature	Transition	Animation (@keyframes)
Trigger	Requires a state change (like hover)	Can run automatically
Control	Start and end state only	Multiple steps defined
Complexity	Simple	Can be complex

Feature	Transition	Animation (@keyframes)
Use case	Button hover, card hover	Loading spinner, fade-in on load

```
/* Transition - smooth color change on hover */
```

```
.button {
  background: #0066cc;
  transition: background 0.3s ease;
}
```

```
.button:hover {
  background: #004499;
}
```

```
/* Animation - runs on its own */
```

```
@keyframes fadeIn {
  from { opacity: 0; }
  to   { opacity: 1; }
}
```

```
.card {
  animation: fadeIn 0.5s ease;
}
```

Q36. How do you add a smooth hover effect to a button using CSS transitions?

Answer:

```
.btn {
  background-color: #e53935;
  color: white;
  padding: 12px 24px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 16px;
}
```

```
/* transition: property duration timing-function */
transition: background-color 0.3s ease, transform 0.2s ease;
}
```

```
.btn:hover {
  background-color: #b71c1c;
  transform: translateY(-2px); /* slight lift effect */
}
```

```
}
```

The **transition** property tells the browser to animate the change smoothly instead of switching instantly.

Transition shorthand:

```
transition: [property] [duration] [timing-function] [delay];
```

Q37. A Swiggy developer wants a card to smoothly grow slightly when hovered. How would they do this?

Answer:

```
.food-card {  
  background: white;  
  border-radius: 12px;  
  padding: 16px;  
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);  
  cursor: pointer;  
  
  /* smooth scale transition */  
  transition: transform 0.3s ease, box-shadow 0.3s ease;  
}  
  
.food-card:hover {  
  transform: scale(1.03); /* grows 3% on hover */  
  box-shadow: 0 8px 24px rgba(0,0,0,0.15); /* deeper shadow */  
}
```

This gives a polished, professional "lift" effect that modern apps use everywhere.

Q38. What is the **transition property shorthand? What are the timing functions?**

Answer:

```
/* Full shorthand */  
transition: property duration timing-function delay;  
  
/* Example */  
transition: all 0.3s ease-in-out 0s;
```

Common timing functions:

Value	Effect
<code>ease</code>	Slow start, fast middle, slow end (default)
<code>linear</code>	Same speed throughout
<code>ease-in</code>	Starts slow, ends fast
<code>ease-out</code>	Starts fast, ends slow
<code>ease-in-out</code>	Slow start and end

```
.box {  
  transition: transform 0.4s ease-out;  
}
```

Q39. What are @keyframes in CSS? Write a simple fade-in animation.

Answer: @keyframes lets you define the exact steps of an animation — from start to finish (or through multiple stages).

```
/* Define the animation steps */  
@keyframes fadeIn {  
  from {  
    opacity: 0;  
    transform: translateY(20px); /* starts slightly below */  
  }  
  to {  
    opacity: 1;  
    transform: translateY(0); /* ends in normal position */  
  }  
}  
  
/* Apply it to an element */  
.hero-section {  
  animation: fadeIn 0.8s ease forwards;  
}
```

animation shorthand:


```
animation: name duration timing-function delay iteration-count direction fill-mode;

/* Common usage */
animation: fadeIn 0.5s ease forwards;
```

Q40. What does **animation-fill-mode: forwards** do?

Answer: By default, after an animation ends, the element goes back to its original CSS state. **forwards** tells the element to **keep the final animation state** after it finishes.

```
@keyframes slideIn {
  from { transform: translateX(-100px); opacity: 0; }
  to   { transform: translateX(0);    opacity: 1; }
}
```

```
/* ❌ Without forwards — element jumps back to invisible after animation */
.card {
  animation: slideIn 0.5s ease;
}
```

```
/* ✅ With forwards — element stays visible after animation ends */
.card {
  animation: slideIn 0.5s ease forwards;
}
```

Interview Tip: **forwards** is almost always what you want for entrance animations like fade-in or slide-in.

Q41. How do you make a simple loading spinner using CSS animations?

Answer:

```
<div class="spinner"></div>

.spinner {
  width: 40px;
  height: 40px;
  border: 4px solid #e0e0e0;
  border-top-color: #0066cc;
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
```

```
}
```

```
@keyframes spin {  
  from { transform: rotate(0deg); }  
  to   { transform: rotate(360deg); }  
}
```

infinite makes the animation loop forever. **linear** keeps the speed constant so the spin looks smooth.

Real use: Loading spinners on Flipkart, Zomato, and Paytm while data is being fetched.

Q42. A developer's media query was not triggering on mobile. What are the common reasons?

Answer:

Reason 1 — Missing viewport meta tag in HTML:

<!-- ❌ Missing — media queries won't work on real mobile devices -->

<!-- ✅ Add this in <head> -->

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Reason 2 — Using wrong condition:

/* ❌ max-width with a too-small value — never triggers */

```
@media (max-width: 100px) { ... }
```

/* ✅ Correct */

```
@media (max-width: 768px) { ... }
```

Reason 3 — Specificity issue — another CSS rule overriding the media query styles

Interview Tip: The missing viewport meta tag is the most common cause of media queries not working on actual mobile devices.

Q43. What is **minmax() in CSS Grid? Give an example.**

Answer: **minmax(min, max)** sets a minimum and maximum size for a grid track (column or row). It prevents columns from getting too small or too large.

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 16px;
}
```

This means: each column is **at least 200px** wide, but can grow up to **1fr** (full available fraction). The grid automatically decides how many columns fit.

Result:

- Desktop → 4 columns (each wider than 200px)
- Tablet → 2 columns
- Mobile → 1 column

All without writing a single media query.

Q44. How would you build a fully responsive job listing page for a site like Naukri using CSS Grid and media queries?

Answer:

```
<div class="job-grid">
  <div class="job-card">Software Engineer - TCS</div>
  <div class="job-card">Frontend Dev - Zoho</div>
  <div class="job-card">UI Developer - Wipro</div>
  <div class="job-card">React Dev - Infosys</div>
</div>
```

```
/* Mobile first - single column */
```

```
.job-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
  padding: 16px;
}
```

```
/* Tablet - 2 columns */
```

```
@media (min-width: 768px) {
  .job-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

```

/* Desktop - 3 columns */
@media (min-width: 1024px) {
  .job-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}

.job-card {
  background: white;
  padding: 20px;
  border-radius: 8px;
  border: 1px solid #ddd;
  transition: box-shadow 0.3s ease;
}

.job-card:hover {
  box-shadow: 0 4px 16px rgba(0,0,0,0.1);
}

```

Q45. What is the difference between `display: none` and `visibility: hidden`?

Answer:

	<code>display: none</code>	<code>visibility: hidden</code>
Element visible?	✗ No	✗ No
Space occupied?	✗ No — removed from layout	✓ Yes — space still reserved
Can animate/transition?	Not easily	Yes

```

/* display: none - element disappears, layout collapses */
.menu {
  display: none;
}

/* visibility: hidden - element invisible but space remains */
.tooltip {
  visibility: hidden;
  transition: visibility 0.3s;
}

.tooltip:hover {

```

```
visibility: visible;
}
```

Real use: Hiding mobile menu with `display: none` on desktop using a media query.

Q46. How would you animate a page header to fade in when the page loads?

Answer:

```
@keyframes fadeDown {
  from {
    opacity: 0;
    transform: translateY(-30px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.page-header {
  animation: fadeDown 0.7s ease forwards;
}

<header class="page-header">
  <h1>Welcome to Our Portal</h1>
</header>
```

The header smoothly fades in from above when the page loads — a professional effect used on government portals and startup landing pages.

Q47. What is `transform` in CSS? How is it used with transitions and animations?

Answer: `transform` lets you visually move, rotate, scale, or skew an element without affecting the page layout.

Common transform functions:

Function	Effect
<code>translateX(20px)</code>	Move right 20px
<code>translateY(-10px)</code>	Move up 10px
<code>scale(1.1)</code>	Grow 10%
<code>rotate(45deg)</code>	Rotate 45 degrees

```

/* Scale on hover */
.card {
  transition: transform 0.3s ease;
}

.card:hover {
  transform: scale(1.05);
}

/* Slide in with animation */
@keyframes slideUp {
  from { transform: translateY(40px); opacity: 0; }
  to   { transform: translateY(0);   opacity: 1; }
}

.card {
  animation: slideUp 0.5s ease forwards;
}

```

Interview Tip: Use `transform` instead of changing `top/left` for animations — it is much smoother and does not cause layout reflow.

Q48. How do you add a delay between animations for multiple cards appearing one after another (staggered animation)?

Answer: Use `animation-delay` on each card with increasing values.

```

@keyframes fadeUp {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}

.card {

```

```

    animation: fadeUp 0.5s ease forwards;
    opacity: 0; /* hidden before animation starts */
}

```

```

.card:nth-child(1) { animation-delay: 0s; }
.card:nth-child(2) { animation-delay: 0.1s; }
.card:nth-child(3) { animation-delay: 0.2s; }
.card:nth-child(4) { animation-delay: 0.3s; }

```

Each card fades in one after the other — a polished effect used on product pages and dashboards.

Q49. A developer built a grid layout but on mobile it was showing 3 tiny columns. How do they fix it?

Answer:

✗ Bad — 3 columns on all screens, breaks on mobile */

```

.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

```

✓ Fix 1 — Media query approach (mobile-first) */

```

.grid {
  display: grid;
  grid-template-columns: 1fr; /* 1 column on mobile */
}

```

```

@media (min-width: 768px) {
  .grid {
    grid-template-columns: repeat(2, 1fr); /* 2 on tablet */
  }
}

```

```

@media (min-width: 1024px) {
  .grid {
    grid-template-columns: repeat(3, 1fr); /* 3 on desktop */
  }
}

```

✓ Fix 2 — Auto-fit with minmax (no media queries needed) */

```

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
}

```

```
}
```

Fix 2 is the most elegant — the grid automatically decides the column count based on available space.

Q50. Build a complete responsive mini layout with a header, 3-column card grid, hover transitions, and a fade-in animation. This is a common take-home or live coding interview task.

Answer:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Jobs Portal</title>
  <style>

    /* Reset */
    * { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      font-family: sans-serif;
      background: #f4f6f9;
      color: #333;
    }

    /* Fade-in animation */
    @keyframes fadeUp {
      from { opacity: 0; transform: translateY(24px); }
      to   { opacity: 1; transform: translateY(0); }
    }

    /* Fixed Header */
    header {
      position: fixed;
      top: 0;
      width: 100%;
      background: #1a237e;
      color: white;
      padding: 16px 32px;
      display: flex;
      justify-content: space-between;
```



```

    align-items: center;
    z-index: 100;
}

/* Main content below fixed header */
main {
    padding: 100px 24px 40px;
    max-width: 1200px;
    margin: 0 auto;
}

/* Responsive Card Grid */
.card-grid {
    display: grid;
    grid-template-columns: 1fr; /* mobile: 1 column */
    gap: 20px;
}

@media (min-width: 768px) {
    .card-grid {
        grid-template-columns: repeat(2, 1fr);
    }
}

@media (min-width: 1024px) {
    .card-grid {
        grid-template-columns: repeat(3, 1fr);
    }
}

/* Cards with transition + staggered animation */
.card {
    background: white;
    border-radius: 12px;
    padding: 24px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.08);
    opacity: 0;
    animation: fadeUp 0.5s ease forwards;
    transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:nth-child(1) { animation-delay: 0s; }
.card:nth-child(2) { animation-delay: 0.1s; }
.card:nth-child(3) { animation-delay: 0.2s; }

.card:hover {
    transform: translateY(-6px);
}

```

```

        box-shadow: 0 8px 24px rgba(0,0,0,0.12);
    }

    .card h3 { margin-bottom: 8px; color: #1a237e; }
    .card p { color: #666; font-size: 14px; }

    .card .apply-btn {
        margin-top: 16px;
        display: inline-block;
        background: #1a237e;
        color: white;
        padding: 10px 20px;
        border-radius: 6px;
        text-decoration: none;
        font-size: 14px;
        transition: background 0.3s ease;
    }

    .card .apply-btn:hover {
        background: #283593;
    }

</style>
</head>
<body>

<header>
    <span>Jobs Portal</span>
    <nav>Home | Jobs | About</nav>
</header>

<main>
    <h2 style="margin-bottom: 24px;">Latest Openings</h2>
    <div class="card-grid">
        <div class="card">
            <h3>Frontend Developer</h3>
            <p>Zoho Corp – Pune, Maharashtra</p>
            <p>₹6-10 LPA | 1-3 years experience</p>
            <a href="#" class="apply-btn">Apply Now</a>
        </div>
        <div class="card">
            <h3>UI Developer</h3>
            <p>Wipro – Bengaluru, Karnataka</p>
            <p>₹5-8 LPA | Freshers welcome</p>
            <a href="#" class="apply-btn">Apply Now</a>
        </div>
        <div class="card">

```

```
<h3>React Developer</h3>
<p>Infosys - Hyderabad, Telangana</p>
<p>₹8-14 LPA | 2-4 years experience</p>
<a href="#" class="apply-btn">Apply Now</a>
</div>
</div>
</main>

</body>
</html>
```